EE-133L PROGRAMMING FUNDAMENTALS

Laboratory Manual

Student Name: Muhammad Umer Bin Qaiser

Registration No. 2025 – EE – 16

Lab Section: (A)

Department of Electrical Engineering
University of Engineering and Technology, Lahore

Experiment No. 5:
Introduction to Functions and Recursive Functions in C Language

Objectives
The objectives of this lab are:

1. To understand the structure, declaration, and usage of functions and recursive functions in C.
2. To learn how to define and call user-defined functions.
3. To understand the concept of function prototypes.
4. To differentiate between Call by Value and Call by Reference.
5. To explore the use of library functions in modular programming.
6. To practice debugging and correction of common errors in function-based programs.

Prerequisites

Following are the prerequisites of this lab:
- Understanding of data types, variables, and operators in C.
- Knowledge of conditional statements (if, if-else, switch).
- Familiarity with loops (for, while, do-while).
- Basic input and output operations using scanf() and printf().

Introduction

Functions
Functions are fundamental building blocks in the C programming language. They allow the programmer to break a large program into smaller, manageable units. Each function performs a specific task, which makes programs easier to develop, debug, and maintain.

Function Categories

In C, functions can be broadly classified into two categories:
Library Functions: Predefined in header files such as <stdio.h>, <math.h>, <string.h>.
User-defined Functions: Created by the programmer for specific needs.
Library functions are ready-to-use, while user-defined functions enhance flexibility and problem-specific behavior.
C provides both library and user-defined functions. Library functions are pre-written and available in standard header files

Syntax of a Function

A function in C consists of a return type, a name, parameters (optional), and a body that defines the statements to be executed. The syntax is as follows:

```
return_type  function_name(parameter_list)
{
   // body of function
   statements;
   return value; // optional
}
```

Function Prototypes

A function prototype tells the compiler about the function's return type, name, and parameter list before its actual definition appears.
It ensures that when the function is called, the compiler knows how many arguments to expect, what types they are, and what type of value the function will return. Using prototypes helps avoid type mismatches and logic errors. Functions can be declared before their use (using prototypes) or defined before main().

Advantages of Using Functions:

1. **Modularity**: Programs can be divided into smaller, independent units.
2. **Reusability**: Functions can be used multiple times in different parts of the program.
3. **Simplicity**: Easier to read, debug, and maintain.
4. **Testing**: Individual modules can be tested independently.
5. **Team Collaboration**: Different programmers can work on different functions simultaneously.

Example 1:

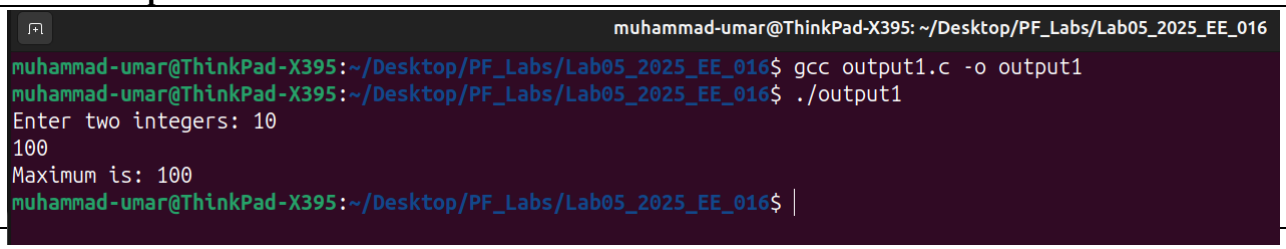
```
#include <stdio.h>
int maximum(int x, int y);  // Function prototype

int main(void)
{
   int num1, num2, result;
   printf("Enter two integers: ");
   scanf("%d %d", &num1, &num2);
   result = maximum(num1, num2);
   printf("Maximum is: %d\n", result);
   return 0;
}
```

```
int maximum(int x, int y)
{
   if (x > y)
      return x;
   else
      return y;
}
```

Expected Output:

```
Enter two integers: 10 25
Maximum is: 25
```

Students' Output:



Example 2:
Program demonstrating Call by Reference using pointers.

```
#include <stdio.h>

void swap(int *a, int *b)
{
   int temp = *a;
   *a = *b;
   *b = temp;
}

int main()
{
    int x = 10, y = 20;
```

```
swap(&x, &y);
   printf("After swapping: x = %d, y = %d\n", x, y);
   return 0;
}
```

Expected Output:

```
After swapping: x = 20, y = 10
```

Student's Output:

```
muhammad-umar@ThinkPad-X395: ~/Desktop/PF_Labs/Lab05_2025_EE_016
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$ gcc output2.c -o output2
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$ ./output2
After swapping: x = 20, y = 10
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$
```

Parameter Passing Mechanisms

C supports two types of parameter passing:

- **Call by Value:** A copy of the actual parameter is passed, so changes made inside the function do not affect the original variable.
- **Call by Reference (simulated using pointers in C):** The address of the variable is passed, allowing direct modification of the original data.

Library (Built-in) Functions

C provides several standard library functions that perform common operations. These functions are defined in header files like <math.h>, <stdio.h>, and <stdlib.h>.
Using library functions saves development time and ensures reliability since they are pre-tested by compiler developers.

Random Number Generation

The C standard library `<stdlib.h>` provides functions for generating pseudo-random numbers. The function `rand()` returns a pseudo-random integer between `0` and `RAND_MAX`. To make the sequence of random numbers less predictable, the `srand()` function can be used to seed the random number generator — typically with a value like `time(NULL)` from `<time.h>`.

Storage Classes and Scope

A variable's storage class determines its lifetime, visibility, and default value.
C provides four main storage classes:

1. **auto:** Default for local variables.
2. **register:** Requests storage in CPU register for faster access.
3. **static:** Retains its value between function calls.
4. **extern:** Refers to a global variable defined elsewhere.

Understanding these helps manage memory and control data visibility effectively.

Debugging Task 1:

The following code is intended to calculate the sum of two numbers using a user-defined function, but it contains multiple syntax errors. Identify and correct them.

```
#include <stdio.h>

int sum(a, b) {
   return a + b
}

int main() {
   int num1 = 5, num2 = 10;
   result = sum(num1 num2);
   printf("Sum = %d", result);
}
```

Corrected Code:

Open debug1.c ~/Desktop/PF_La... b05_2025_EE_016

```
#include <stdio.h>
int sum(int a, int b) {
return a + b;
}
int main() {
int num1 = 5, num2 = 10, result;
result = sum(num1, num2);
printf("Sum = %d", result);
}
```

Output:

```
muhammad-umar@ThinkPad-X395: ~/Desktop/PF_Labs/Lab05_2025_EE_016
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$ make debug1
gcc -c debug1.c
gcc debug1.o -o debug1
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$ ./debug1
Sum = 15muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$
```

Lab Tasks

Task 1

Write a program that defines a function float areaCircle(float r) to calculate the area of a circle.

Code:

circle_area.c
~/Desktop/PF_Labs/Lab05_2025_EE_016

```c
#include <stdio.h>
#define PI (float) 22 / (float) 7
float areaCircle(float r);
int main(){
float rad, area;
printf("Enter the radius: ");
scanf("%f", &rad);

area = areaCircle(rad);
printf("Area: %.2f\n", area);
}

float areaCircle(float r){

return PI * r * r;

}
```

Output:

```
muhammad-umar@ThinkPad-X395: ~/Desktop/PF_Labs/Lab05_2025_EE_016
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$ make circle_area
gcc -c circle_area.c
gcc circle_area.o -o circle_area
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$ ./circle_area
Enter the radius: 4
Area: 50.29
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$
```

Task 2

Write a function int factorial(int n) that computes factorial using iteration.

Code:

factorial.c
~/Desktop/PF_Labs/Lab05_2025_EE_016

```c
#include <stdio.h>

int factorial(int n);
int main(){
int num, fact;
printf("Enter a number: ");
scanf("%d", &num);

fact = factorial(num);
printf("Factorial: %d\n", fact);
}

int factorial(int n){
int fact = 1;
for (int i = 1; i <= n; i++){
fact *= i;
}

return fact;

}
```

Output:

```
muhammad-umar@ThinkPad-X395: ~/Desktop/PF_Labs/Lab05_2025_EE_016
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$ make factorial
gcc -c factorial.c
gcc factorial.o -o factorial
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$ ./factorial
Enter a number: 6
Factorial: 720
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$
```

Task 3

Write a program to input marks of students and find the average using two functions:

- inputMarks()
- averageMarks()

Code:

marks.c
~/Desktop/PF_Labs/Lab05_2025_EE_016

```c
#include <stdio.h>

int inputmarks(int students);
float averagemarks(int total, int students);

int main(){
int students, sum;
float average;
printf("Enter the number of Students: ");
scanf("%d", &students);

sum = inputmarks(students);
average = averagemarks(sum, students);

printf("Class Average: %.2f\n", average);
}

int inputmarks(int students){
int total = 0, current;
for (int i = 1; i <= students; i++){
printf("Enter marks of Student # %d: ", i);
scanf("%d", &current);
total += current;
}
return total;
}

float averagemarks(int total, int students){
return (float) total / (float) students;
}
```

Output:

```
muhammad-umar@ThinkPad-X395: ~/Desktop/PF_Labs/Lab05_2025_EE_016
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$ make marks
gcc -c marks.c
gcc marks.o -o marks
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$ ./marks
Enter the number of Students: 3
Enter marks of Student # 1: 77
Enter marks of Student # 2: 97
Enter marks of Student # 3: 98
Class Average: 90.67
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$
```

Task 4

Write a menu-driven calculator program that uses functions for addition, subtraction, multiplication, and division.

Code:

```
Open  calculator.c
~/Desktop/PF_Labs/Lab05_2025_EE_016
#include <stdio.h>
float add(float x, float y){
        return (x + y);}
float sub(float x, float y){
        return (x - y);}
float mult(float x, float y){
        return (x * y);}
float div(float x, float y){
        if (y == 0) {fprintf(stderr, "Cannot Divide By
Zero!");
        return 0;}
        return (x / y);}
int main(){
float A, B;
int choice;
printf("Enter A: ");
scanf("%f", &A);
printf("Enter B: ");
scanf("%f", &B);
printf("\t\t----MENU----\n\n1. Addition\n2. Subtraction\n3.
Multiplication\n4. Division\n\nEnter the Corresponding
Number: ");
scanf("%d", &choice);
```

```
switch (choice){

case 1:
        printf("%.2f\n", add(A, B));
        break;
case 2:
        printf("%.2f\n", sub(A, B));
        break;
case 3:
        printf("%.2f\n", mult(A, B));
        break;
case 4:
        printf("%.2f\n", div(A, B));
        break;
default:
        printf("Invalid Choice!!!\n");
        break;
}

}
```

Output:

```
muhammad-umar@ThinkPad-X395: ~/Desktop/PF_Labs/Lab05_2025_EE_016
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$ make calculator
gcc -c calculator.c
gcc calculator.o -o calculator
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$ ./calculator
Enter A: 2
Enter B: 2
                ----MENU----

1. Addition
2. Subtraction
3. Multiplication
4. Division

Enter the Corresponding Number: 1
4.00
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$
```

Debugging Task 2:

The following code is supposed to compute the cube of a number, but it contains several syntax and logic errors. Fix them and execute successfully.

```
#include <stdio.h>

void cube(num) {
   return num * num * num;
}

int main(void) {
   int n;
   printf("Enter number: ")
   scanf("%d", n);
   printf("Cube is: %d", cube(n));
   return 0
}
```

Corrected code:

debug2.c
~/Desktop/PF_La... 05_2025_EE_016

```c
#include <stdio.h>
int cube(int num) {
return num * num * num;
}
int main(void) {
int n;
printf("Enter a number: ");
scanf("%d", &n);
printf("Cube is: %d", cube(n));
return 0;
}
```

Output:

muhammad-umar@ThinkPad-X395: ~/Desktop/PF_Labs/Lab05_2025_EE_016

```
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$ make debug2
gcc -c debug2.c
gcc debug2.o -o debug2
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$ ./debug2
Enter a number: 4
Cube is: 64muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$
```

Introduction to Recursive Functions in C Language

Introduction

Recursion

Recursion is a programming technique where a function calls itself to solve a problem by breaking it into smaller subproblems of the same type. It is a powerful alternative to iteration and is especially useful for problems with a naturally recursive structure (e.g., tree traversal, factorial, Fibonacci).

A recursive function must have:

• Base Case: A condition that stops the recursion.
• Recursive Case: The part where the function calls itself with a smaller input.

Without a proper base case, recursion leads to infinite calls and stack overflow.

Types of Recursion

In C, Recursion functions can be broadly classified into two categories:

1. **Direct Recursion**
   When the recursion functions in C call themselves directly, it is known as direct recursion.

2. **Indirect Recursion**
   When a function calls another function, and that function again calls the first function, it's called indirect recursion.

3. **Tail Recursion**
   A recursion is called tail recursion if the recursive call is the last statement executed by the function.

4. **Head Recursion**
   If the recursive call happens before any other processing in the function, it is known as head recursion.

5. **Tree Recursion**
   When a function calls itself more than once in each invocation, it results in multiple branches, forming a tree-like structure.

How Recursion Works

When a function calls itself:
1. A new stack frame is pushed onto the call stack.
2. Local variables and parameters are stored in this frame.
3. Execution pauses until the recursive call returns.
4. After return, the frame is popped, and execution resumes

Syntax of a Recursive Function

A **recursive function** is simply a function that **calls itself** inside its own body.
The syntax looks like this:

```
returnType functionName(parameters) {
    if (base_case_condition) {
        // Base Case: defines when the recursion should stop
        return value;
    } else {
        // Recursive Case: function calls itself with modified parameters
        return functionName(updated_parameters);
    }
}
```

Why Recursion used in C?

Recursion is used to perform complex tasks such as tree and graph structure traversals. Popular recursive programming solutions include factorial, binary search, tree traversal, tower of Hanoi, eight queens problem in chess, etc
A recursive program becomes concise, it is not easily comprehendible. Even if the size of the code may reduce, it needs more resources of the processor, as it involve multiple IO calls to the functions.

Advantages of Using Recursive Functions:

- **Simplicity:** Code gets cleaner and uses fewer needless function calls.
- **Elegant solution:** Helpful for resolving difficult algorithms and formula-based challenges.
- **Tractable approach**: They are useful for traversing trees and graphs because they are naturally recursive.
- **Code reusability:** Recursive functions in C can be used repeatedly within the program for different inputs, promoting code reusability.

Disadvantages of Using Recursive Functions:

- **Complexity:** It gets challenging to read and decipher the code.
- Stack consumption: The copies of recursive functions take up a significant amount of memory.
- **Performance:** Recursive calls can be slower and less efficient than iterative solutions due to function call overhead and repeated calculations.
- **Limited applicability:** Not all problems are well-suited for C recursion, and converting iterative solutions into recursive ones may not always be practical or beneficial.

Example 1:

```c
#include <stdio.h>

long long factorial(int n);  // Function prototype

int main(void) {
    int num;
    printf("Enter a non-negative integer: ");
    scanf("%d", &num);

    if (num < 0)
        printf("Factorial is not defined for negative numbers.\n");
    else
        printf("Factorial of %d = %lld\n", num, factorial(num));

    return 0;
}
```

```
long long factorial(int n) {
    if (n == 0 || n == 1)        // Base case
        return 1;
    else
        return n * factorial(n - 1);  // Recursive case
}
```

Expected Output:

```
Enter a non-negative integer: 6
Factorial of 6 = 720
```

Students' Output:

```
muhammad-umar@ThinkPad-X395: ~/Desktop/PF_Labs/Lab05_2025_EE_016
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$ gcc output3.c -o output3
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$ ./output3
Enter a non-negative integer: 4
Factorial of 4 = 24
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$
```

Example 2:
Recursive Fibonacci

```
#include <stdio.h>

int fibonacci(int n);  // Prototype

int main() {
    int n, i;
    printf("Enter number of terms: ");
    scanf("%d", &n);

    printf("Fibonacci Series: ");
    for (i = 0; i < n; i++) {
        printf("%d ", fibonacci(i));
```

```
    }
    printf("\n");
    return 0;
}

int fibonacci(int n) {
    if (n == 0) return 0;           // Base case 1
    if (n == 1) return 1;           // Base case 2
    return fibonacci(n-1) + fibonacci(n-2);  // Recursive
}
```

Expected Output:

Enter number of terms: 3
Fibonacci Series: 0 1 1

Student's Output:

```
muhammad-umar@ThinkPad-X395: ~/Desktop/PF_Labs/Lab05_2025_EE_016
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$ gcc output4.c -o output4
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$ ./output4
Enter number of terms: 5
Fibonacci Series: 0 1 1 2 3
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$
```

Debugging Task 3:

The following code is intended to compute the sum of digits recursively but contains multiple errors. Identify and correct them.

```
#include <stdio.h>

int sumDigits(n)
{
  if (n = 0)
    return 0;
  else
    return n % 10 + sumDigits(n / 10)
}

int main()
```

```
{
   int num;
   printf("Enter number: ");
   scanf("%d", &num);

   printf("Sum of digits = %d", sumDigits(num));
}
```

Corrected Code:

```
#include <stdio.h>
int sumDigits(int n)
{
if (n == 0) return 0;
else return n % 10 + sumDigits(n / 10);
}
int main(){
int num;
printf("Enter number: ");
scanf("%d", &num);
printf("Sum of digits = %d", sumDigits(num));
}
```

Output:

```
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$ make debug3
gcc -c debug3.c
gcc debug3.o -o debug3
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$ ./debug3
Enter number: 64
Sum of digits = 10muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$
```

Lab Tasks

Task 5

Write a recursive function int power(int base, int exp) to compute:

Code:

```
Open  • recur_exp.c
~/Desktop/PF_Labs/Lab05_2025_EE_016

#include <stdio.h>
int recursive_exponent(int base, int exp){
if (exp == 0) return 1;
if (exp == 1) return base;
return base * (recursive_exponent(base, exp - 1));

}
int main(){
int base, exp, solution;
printf("Enter Base: ");
scanf("%d", &base);
printf("Enter Exponent: ");
scanf("%d", &exp);
solution = recursive_exponent(base, exp);

printf("Solution : %d\n", solution);

}
```

Output:

```
muhammad-umar@ThinkPad-X395: ~/Desktop/PF_Labs/Lab05_2025_EE_016
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$ make recur_exp
gcc -c recur_exp.c
gcc recur_exp.o -o recur_exp
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$ ./recur_exp
Enter Base: 5
Enter Exponent: 3
Solution : 125
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$
```

Task 6

Write a recursive function int sum(int n) that returns the sum of first **n** natural numbers.

Code:

recur_sum.c
~/Desktop/PF_Labs/Lab05_2025_EE_016

```c
#include <stdio.h>
int recur_sum(int n){
if (n == 0) return 0;
return n + recur_sum(n - 1);
}
int main(){
int num;
printf("Enter Number: ");
scanf("%d", &num);

printf("Sum = %d\n", recur_sum(num));


}
```

Output:

muhammad-umar@ThinkPad-X395: ~/Desktop/PF_Labs/Lab05_2025_EE_016

```
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$ make recur_sum
gcc -c recur_sum.c
gcc recur_sum.o -o recur_sum
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$ ./recur_sum
Enter Number: 10
Sum = 55
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$
```

Task 7

Write a program that uses a recursive function to find the **GCD** of two numbers.

Code:

Open • GCD.c
~/Desktop/PF_Labs/Lab05_2025_EE_016

```c
#include <stdio.h>

int recursive_gcd(int a, int b){

if (b == 0) return a;

return recursive_gcd(b, a%b);

}

int main(){
int a, b;
printf("Enter A: ");
scanf("%d", &a);
printf("Enter B: ");
scanf("%d", &b);

printf("GCD: %d\n", recursive_gcd(a, b));
}
```

Output:

muhammad-umar@ThinkPad-X395: ~/Desktop/PF_Labs/Lab05_2025_EE_016

```
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$ make GCD
gcc -c GCD.c
gcc GCD.o -o GCD
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$ ./GCD
Enter A: 5
Enter B: 25
GCD: 5
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$
```

Task 8

Write a recursive function void printReverse(int n) that prints digits of a number in reverse order.

Code:

```
#include <stdio.h>
void recursive_reverse(int n){
if (n == 0){printf("\n"); return;}
printf("%d",n % 10);
return recursive_reverse(n / 10);
}
int main(){
int n;

printf("Enter number: ");
scanf("%d", &n);

recursive_reverse(n);

}
```

Output:

```
muhammad-umar@ThinkPad-X395: ~/Desktop/PF_Labs/Lab05_2025_EE_016
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$ make recursive_reverse
gcc -c recursive_reverse.c
gcc recursive_reverse.o -o recursive_reverse
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$ ./recursive_reverse
Enter number: 1234
4321
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$
```

Debugging Task 4:

The following program is supposed to **count the number of digits** in a number using a **recursive function**, but it contains several syntax and logical errors.
Identify the mistakes and correct them.

```
#include <stdio.h>

int countDigits(n)
{
   if(n == 1)
      return 1;
   else
      return 1 + countDigits(n - 10);
}

int main()
{
   int num;
   printf("Enter a number: ")
   scanf("%d", num);

   digits = countDigits();

   printf("Total digits are: %d", digits)
}
```

Corrected code:

debug4.c
~/Desktop/PF_Labs/Lab05_2025_EE_016

```
#include <stdio.h>
int countDigits(int n)
{
if(n == 1)
return 1;
else
return 1 + countDigits(n / 10);
}
int main()
{
int num, digits;
printf("Enter a number: ");
scanf("%d", &num);
digits = countDigits(num);
printf("Total digits are: %d", digits);
}
```

Output:

```
muhammad-umar@ThinkPad-X395: ~/Desktop/PF_Labs/Lab05_2025_EE_016
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$ make debug4
gcc -c debug4.c
gcc debug4.o -o debug4
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$ ./debug4
Enter a number: 124124
Total digits are: 6muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab05_2025_EE_016$
```